Padrões GoF – Factory Method

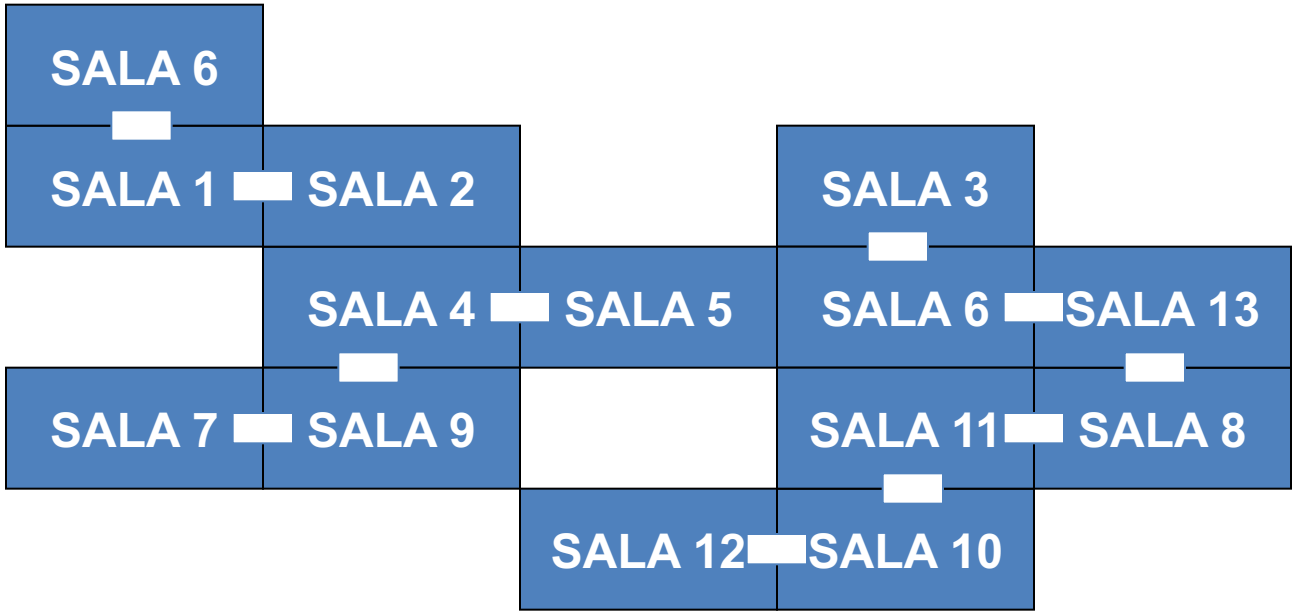
Sabrina de F. Souto.

ROTEIRO

- Construindo um **Jogo do Labirinto**
- Padrão de Criação **Factory Method**
  - **Exemplo: Jogo do Labirinto**

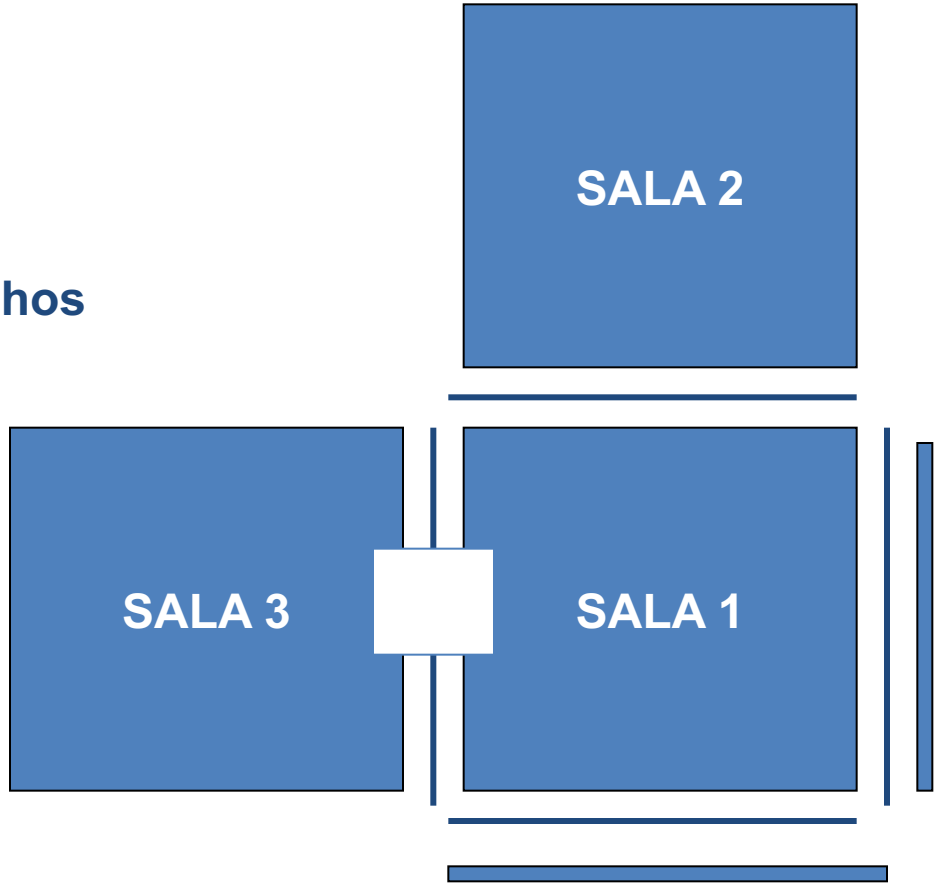
JOGO DO LABIRINTO

Conjunto de SALAS



JOGO DO LABIRINTO

SALA
Conhece seus 4 vizinhos



PORTA PARA
OUTRA SALA

PAREDE

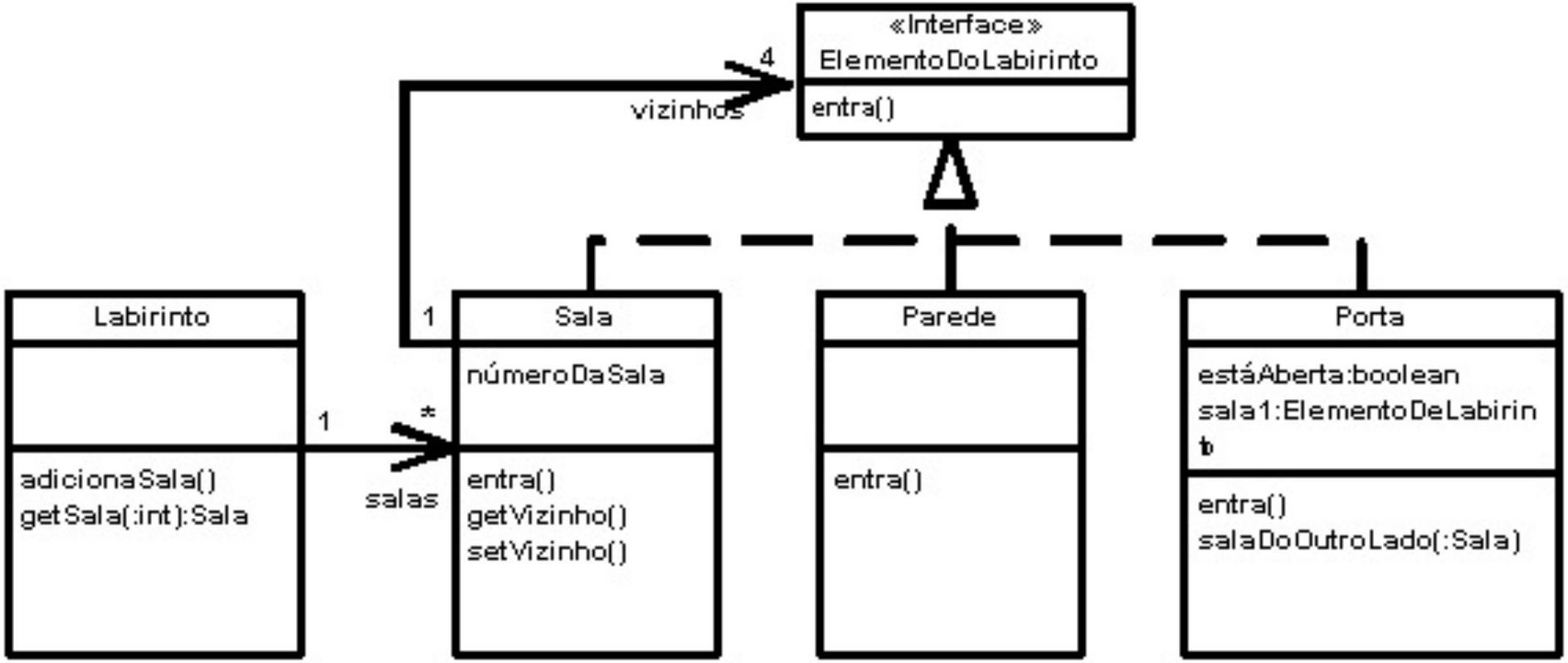
PAREDE

JOGO DO LABIRINTO

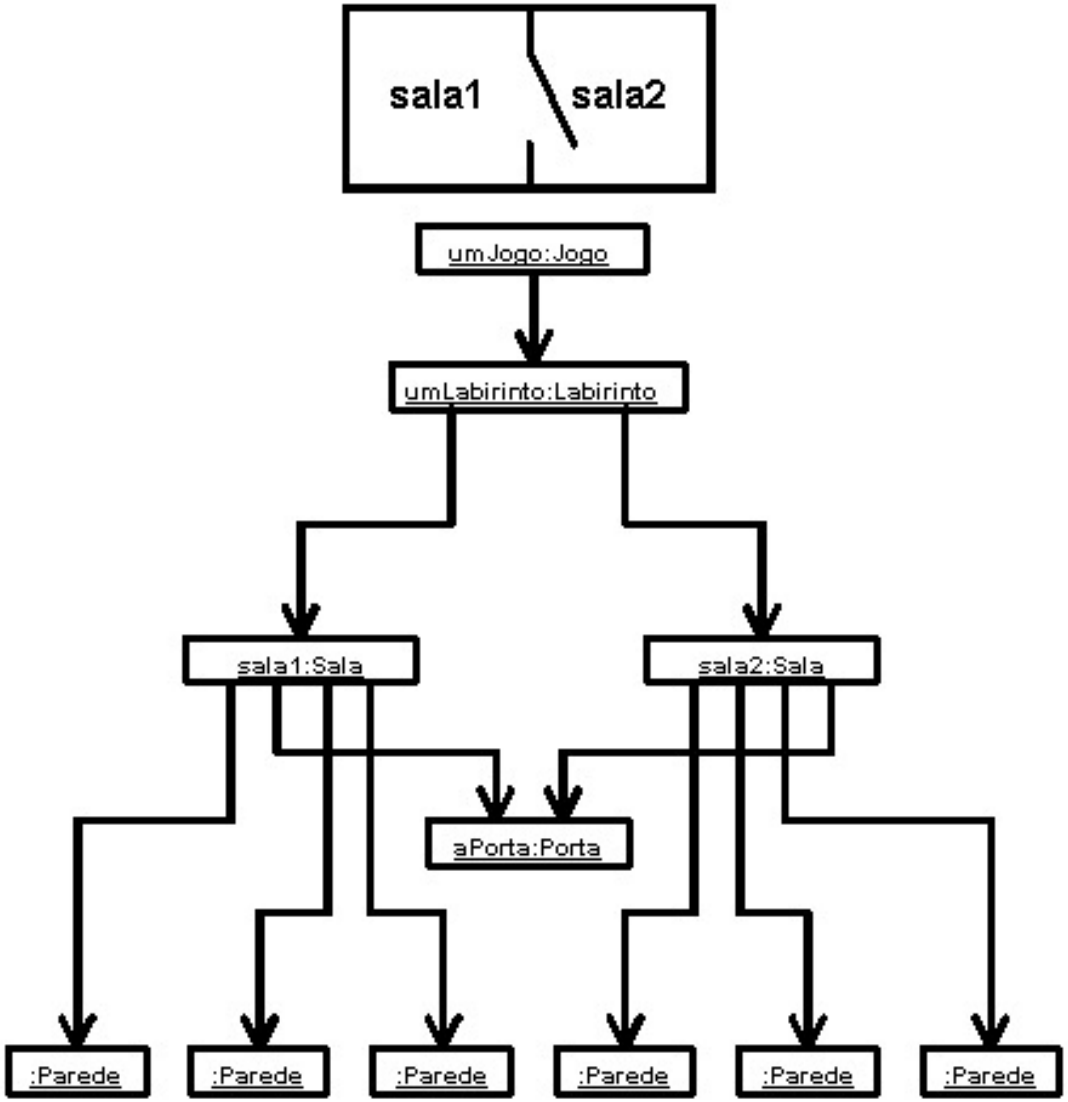
- Como criar este labirinto?
  - É a nossa missão de hoje!

- Quais são as entidades existentes?
  - Sala
  - Porta
  - Parede

JOGO DO LABIRINTO – UML

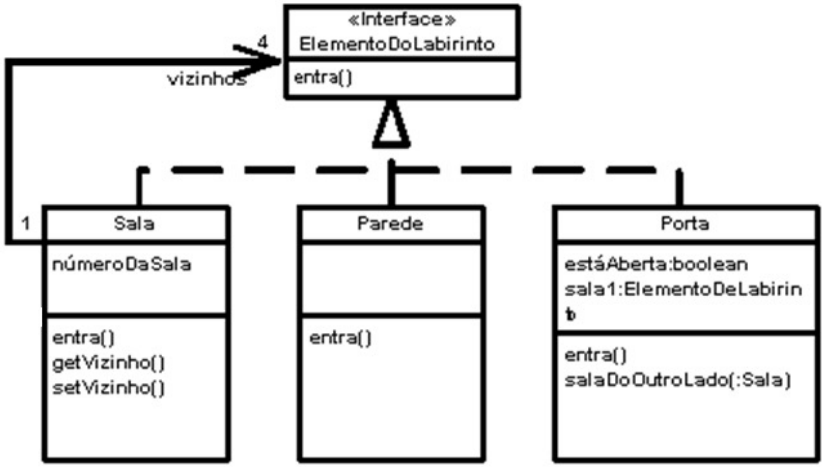


JOGO DO LABIRINTO – DIAGRAM DE OBJETOS



JOGO DO LABIRINTO

- A sala tem 4 vizinhos
  - Norte
  - Sul
  - Leste
  - Oeste
- A interface ElementoDeLabirinto é implementada por todos os elementos do labirinto
  - Sala
  - Porta
  - Parede



JOGO DO LABIRINTO

- Interface ElementoDeLabirinto tem o método **entra()**
  - Sala:
    - a localização do jogador muda
  - Porta:
    - Se aberta: o jogador vai para outra sala
    - Se fechada: o jogador se machuca
  - Parede:
    - o jogador se machuca

- Interface ElementoDeLabirinto tem o método **entra()**

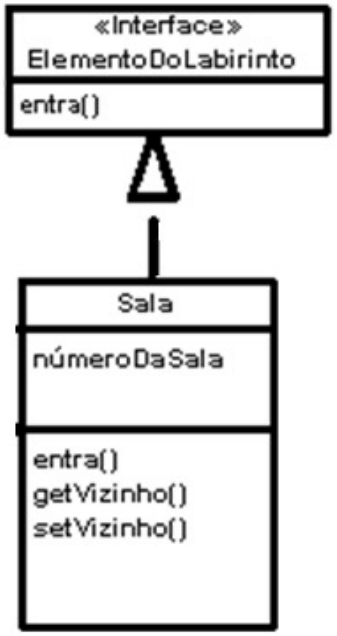
```
public interface ElementoDeLabirinto {
  public void entra();
}
```

JOGO DO LABIRINTO

- Exemplo
  - Se você estiver numa sala e quiser implementar a operação "vá para o Leste", o jogo determina qual ElementoDeLabirinto está do lado Leste e chama entra() deste objeto
    - O método entra() da subclasse específica determina o que ocorre

JOGO DO LABIRINTO

- Classe Sala
  - Implementa ElementoDeLabirinto
  - Define as relações-chave entre objetos
    - Mantém referências para 4 outros ElementoDeLabirinto
  - Armazena um número de sala para identificar as salas do labirinto



JOGO DO LABIRINTO

- Classe **Sala**

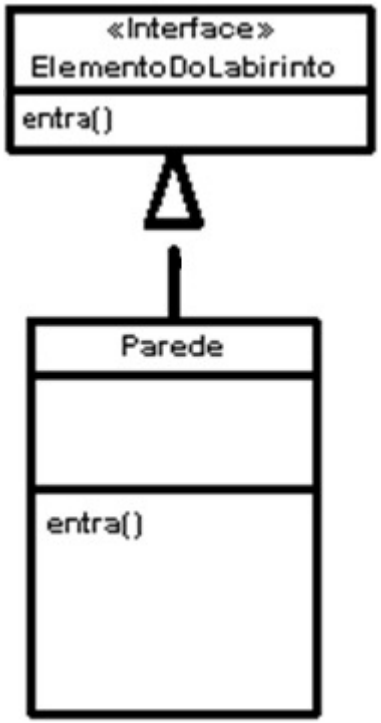
```
public class Sala implements ElementoDeLabirinto {
  private ElementoDeLabirinto[] vizinhos =
          new ElementoDeLabirinto[4];
  private int numeroDaSala;

  public Sala(int númeroDaSala) {
    ...
  }
  public void entra() {
    ...
  }
  public ElementoDeLabirinto getVizinho(int direção) {
    ...
  }
  public void setVizinho(int direção,
                         ElementoDeLabirinto vizinho) {
    ...
  }
}
```

JOGO DO LABIRINTO

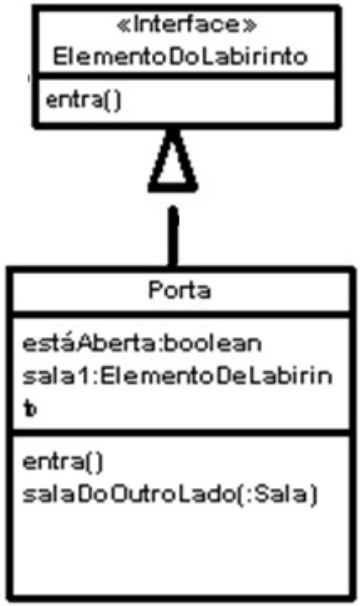
- Classe **Parede**

```
public class Parede implements ElementoDeLabirinto {
  public Parede() {
    ...
  }
  public void entra() {
    ...
  }
}
```



JOGO DO LABIRINTO

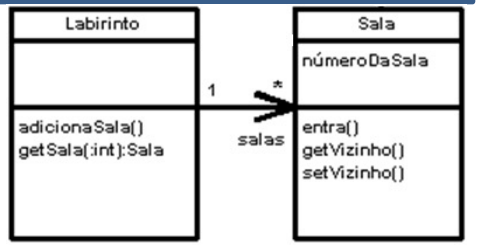
- Classe **Porta**




```
public class Porta implements ElementoDeLabirinto {
  private ElementoDeLabirinto sala1, sala2;
  private boolean estáAberta;

  public Porta(ElementoDeLabirinto sala1,
               ElementoDeLabirinto sala2) {
    ...
  }
  public void entra() {
    ...
  }
  public ElementoDeLabirinto salaDoOutroLado(ElementoDeLabirinto sala) {
      ...
  }
}
```

JOGO DO LABIRINTO



- Classe **Labirinto**
  - Representa uma coleção de salas
  - Recupera uma sala a partir de seu número

```
public class Labirinto {
  private Collection salas = new Vector();

  public Labirinto() {
    ...
  }
  public void adicionaSala(ElementoDeLabirinto sala) {
    ...
  }
  public ElementoDeLabirinto getSala(int númeroDaSala) {
    ...
  }
}
```

JOGO DO LABIRINTO

- Classe Jogo - Main
  - Cria os elementos
  - Conecta os elementos

- Exemplo:
  - Labirinto com 2 salas e uma porta entre elas

JOGO DO LABIRINTO

```
public class Jogo {
  ...
  public Labirinto montaLabirinto() {
    Labirinto umLabirinto = new Labirinto();
    Sala sala1 = new Sala(1);
    Sala sala2 = new Sala(2);
    Porta aporta = new Porta(sala1,sala2);

    umLabirinto.adicionaSala(sala1);
    umLabirinto.adicionaSala(sala2);

    sala1.setVizinho(NORTE, new Parede());
    sala1.setVizinho(LESTE, aporta);
    sala1.setVizinho(SUL, new Parede());
    sala1.setVizinho(OESTE, new Parede());

    sala2.setVizinho(NORTE, new Parede());
    sala2.setVizinho(LESTE, new Parede());
    sala2.setVizinho(SUL, new Parede());
    sala2.setVizinho(OESTE, aporta);

    return umLabirinto;
  }
  ...
}
```

Criando os objetos

Inserindo as salas no labirinto

Conectando os elementos Para a sala 1

Conectando os elementos Para a sala 2

JOGO DO LABIRINTO

- Esta solução tem problemas?
- Qual (is)?
  - Solução inflexível
    - O método **montaLabirinto()** não é reutilizável em outras situações
      - Ele sempre vai criar um labirinto com 2 salas e uma porta entre estas

JOGO DO LABIRINTO

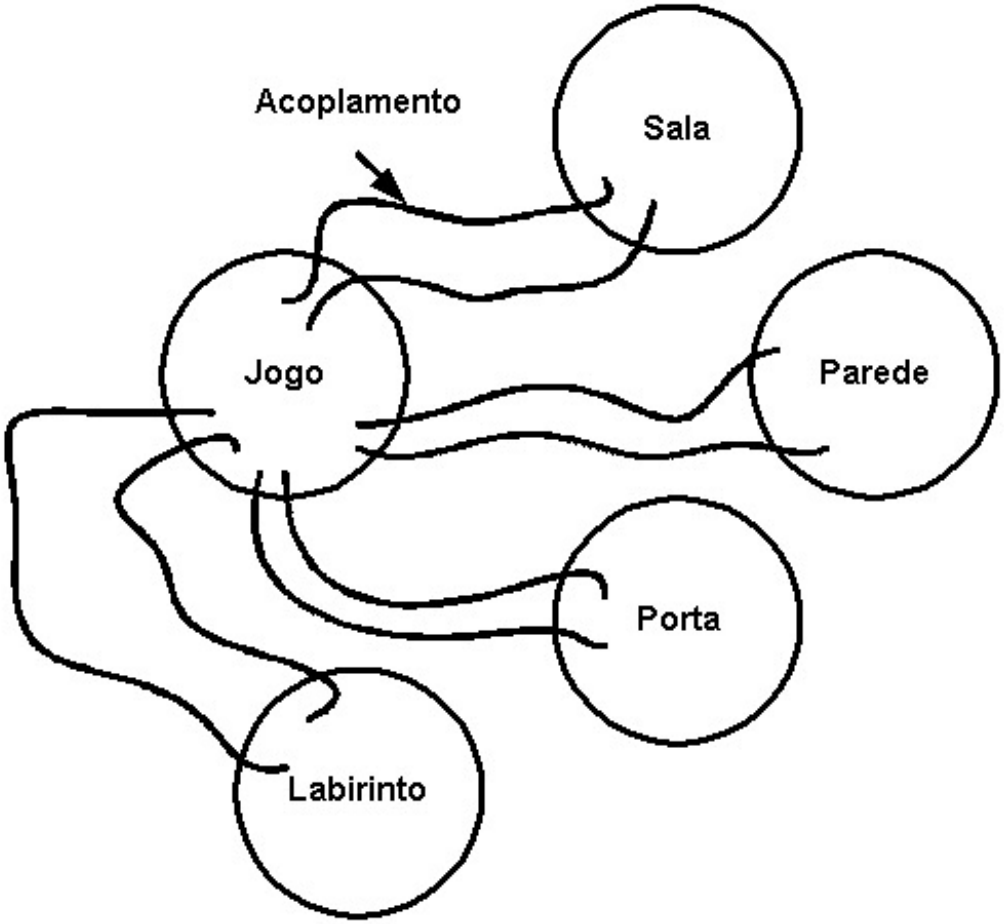
- E se quisermos outro tipo de labirinto?
  - Labirintos encantados
    - Com portas travadas que precisam de um encantamento para abrir
    - Salas contendo encantamentos que podem ser apanhados
  - Labirintos perigosos
    - Salas com bombas que podem ser explodidas para danificar as paredes (e talvez o jogador!)

JOGO DO LABIRINTO

- Problema:
  - O método **montaLabirinto()** mistura a estrutura do labirinto com os tipos dos objetos que compõe o labirinto
- A classe Jogo conhece
  - Sala
  - Parede
  - Porta
- O **new** cria um forte acoplamento entre as classes
  - Define uma amarração com relação ao tipo de objeto que está sendo criado
    - amarra a implementação!

JOGO DO LABIRINTO

- Acoplamento da classe Jogo



JOGO DO LABIRINTO

- Buscando outra solução...
  - Como **alterar montaLabirinto()** para facilmente criar estes novos tipos de labirintos?
  - O maior problema é que a solução atual nos **força a colocar em código as classes concretas que serão instanciadas**
  - **Usaremos padrões de criação para tornar o projeto mais flexível (mais reusável)**

Padrão de Criação

FACTORY METHOD

FACTORY METHOD

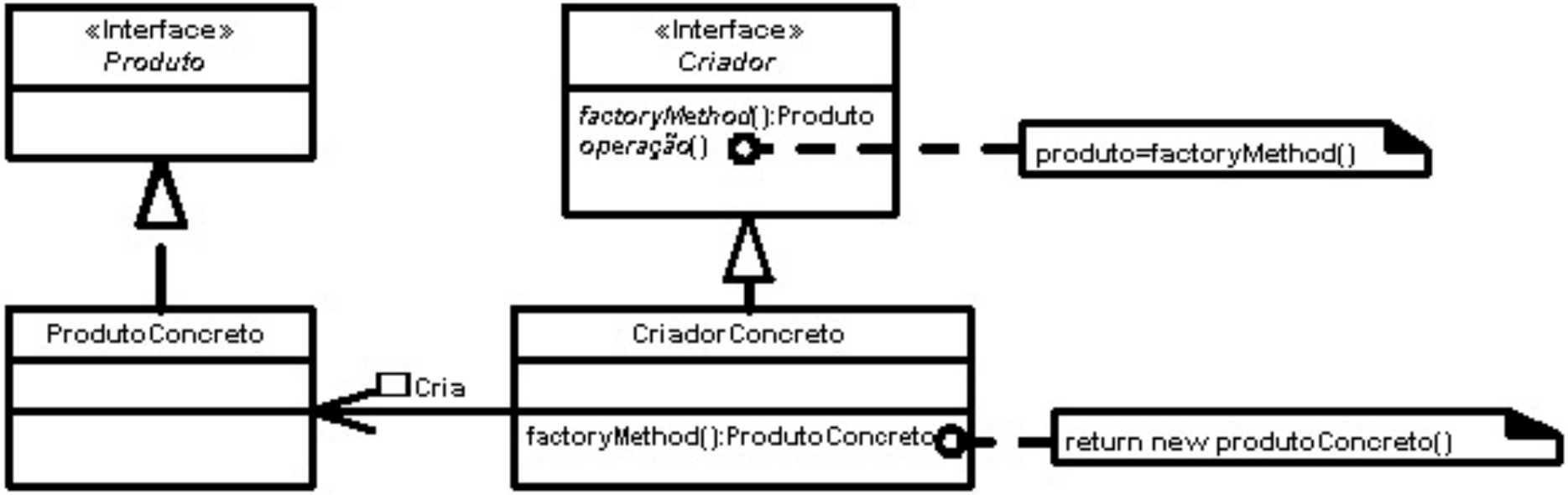
- **Objetivo**
  - Definir uma interface para criar objetos
    - de forma a deixar subclasses clientes decidirem qual classe instanciar
  - Deixar que subclasses façam a instanciação

FACTORY METHOD

- Resumo
  - O cliente (Jogo) precisa **instanciar classes concretas** (Sala, Parede, Porta)
  - O cliente (Jogo) **não chama o construtor** (new) das classes concretas (Sala, Parede, Porta)
  - O cliente chama um **método abstrato (Factory Method)** especificado em alguma classe abstrata (ou interface)
  - Subclasses da classe abstrata podem modificar o tipo de classe concreta a ser retornada a partir do Factory Method

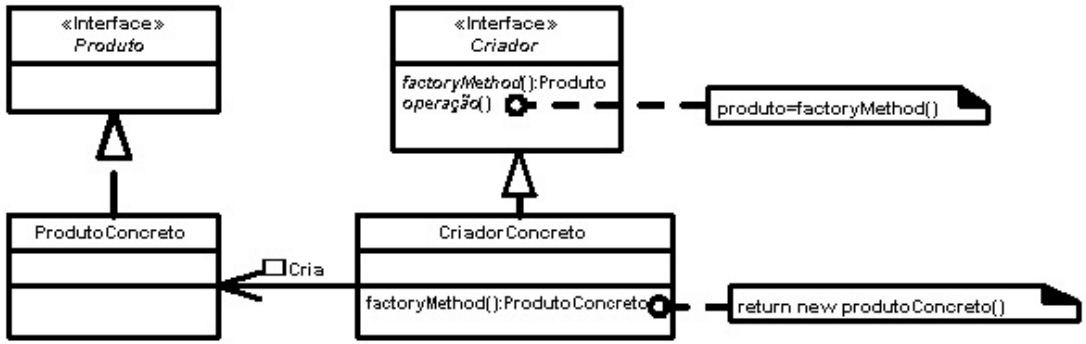
FACTORY METHOD

- **Estrutura genérica**



FACTORY METHOD

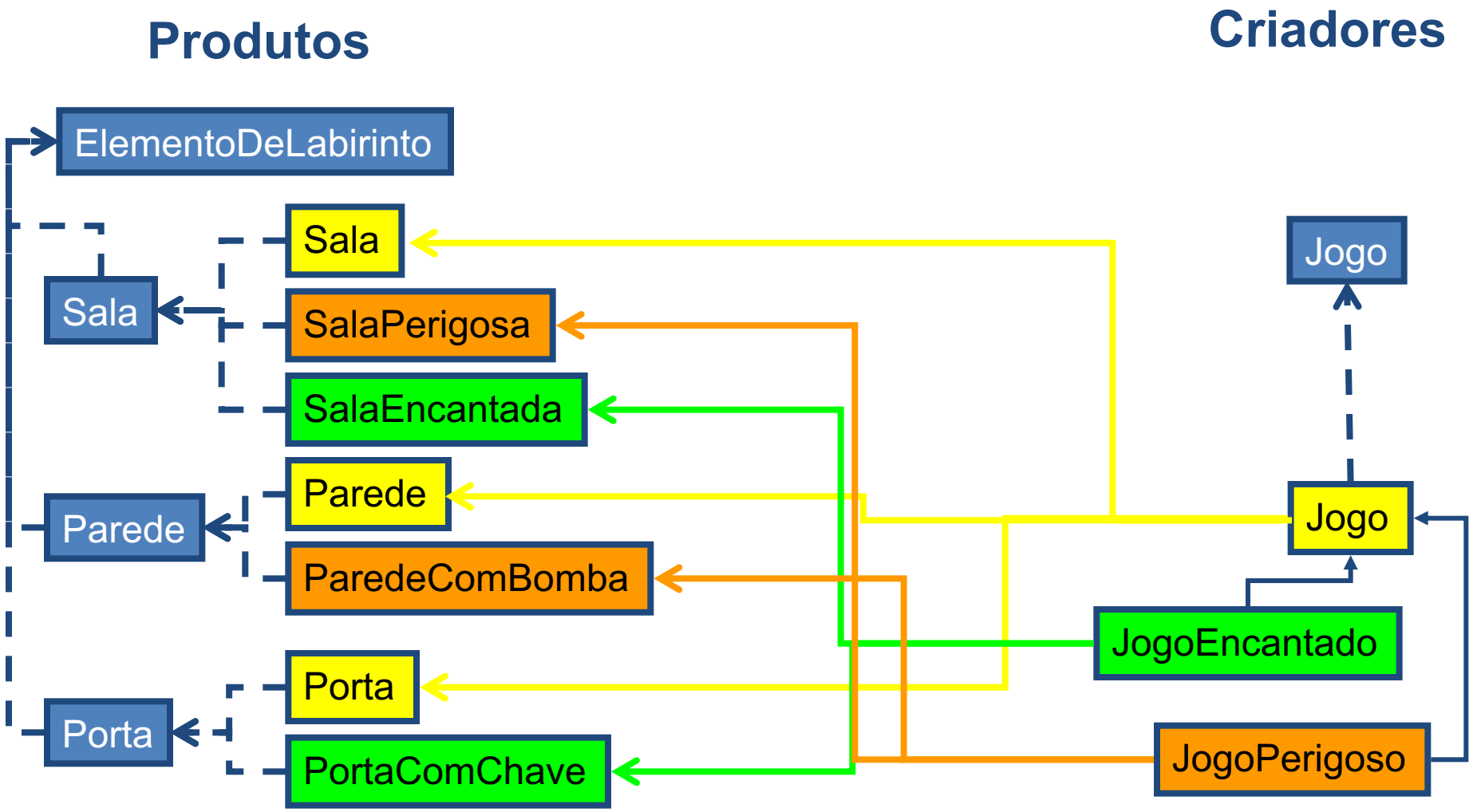
- **Participantes**
  - **Produto**: define a interface dos objetos criados pelo Factory Method
  - **ProdutoConcreto**: implementa a interface Produto
  - **Criador**: declara os Factory Methods que retornam um objeto do tipo Produto
    - Poder ser uma interface
    - Pode ser uma classe com implementação *default* para os Factory Methods
  - **CriadorConcreto**: faz *override* do Factory Method para retornar uma instância de ProdutoConcreto específico



FACTORY METHOD

- **Nosso exemplo**
  - **Produto** → Interfaces
    - Sala, Parede, Porta
  - **ProdutoConcreto** → Classes
    - Sala, SalaEncantada, SalaPerigosa, Parede, ParedeComBomba, Porta, PortaComChave
  - **Criador** → Classe (para o nosso exemplo)
    - Jogo
      - Seu método montaLabirinto() cria o labirinto chamando Factory Methods
      - Ele também é um CriadorConcreto pois oferece uma implementação default para os Factory Methods (para criar um labirinto simples)
  - **CriadorConcreto** → Subclasses do Criador
    - Jogo, JogoEncantado, JogoPerigoso

JOGO DO LABIRINTO – NOVA SOLUÇÃO



JOGO DO LABIRINTO – NOVA SOLUÇÃO

```
public class Jogo {
  // Factory Methods com default
  public Labirinto criaLabirinto() {
    return new Labirinto();
  }
  public Sala criaSala(int númeroDaSala)
    return new Sala(numeroDaSala);
  }
  public Parede criaParede() {
    return new Parede();
  }
  public Porta criaPorta(Sala sala1, Sal
    return new Porta(sala1, sala2);
  }
```

```
  public Labirinto montaLabirinto() {
    Labirinto umLabirinto = criaLabirinto();
    Sala sala1 = criaSala(1);
    Sala sala2 = criaSala(2);
    Porta aPorta = criaPorta(sala1, sala2);

    umLabirinto.adicionaSala(sala1);
    umLabirinto.adicionaSala(sala2);

    sala1.setVizinho(NORTE, criaParede());
    sala1.setVizinho(LESTE, aporta);
    sala1.setVizinho(SUL, criaParede());
    sala1.setVizinho(OESTE, criaParede());

    sala2.setVizinho(NORTE, criaParede());
    sala2.setVizinho(LESTE, criaParede());
    sala2.setVizinho(SUL, criaParede());
    sala2.setVizinho(OESTE, aporta);

    return umLabirinto;
  }
}
```

JOGO DO LABIRINTO – NOVA SOLUÇÃO

- E se quisermos um **Jogo Perigoso**?

```
public class JogoPerigoso extends Jogo {
  public Parede criaParede() {
    return new ParedeDestrutível();
  }
  public Sala criaSala(int númeroDaSala) {
    return new SalaComBomba(númeroDaSala);
  }
}
```

JOGO DO LABIRINTO – NOVA SOLUÇÃO

- E se queremos um **Jogo Encantado**?

```
// um novo CriadorConcreto
public class JogoEncantado extends Jogo {
  public Sala criaSala(int numeroDaSala) {
    return new salaEncantada(númeroDaSala, jogaEncantamento());
  }
  public Porta criaPorta(Sala sala1, Sala sala2) {
    return new portaPrecisandoDeEncantamento(sala1, sala2);
  }
  protected Encantamento jogaEncantamento() {
    ...
  }
}
```

JOGO DO LABIRINTO – NOVA SOLUÇÃO

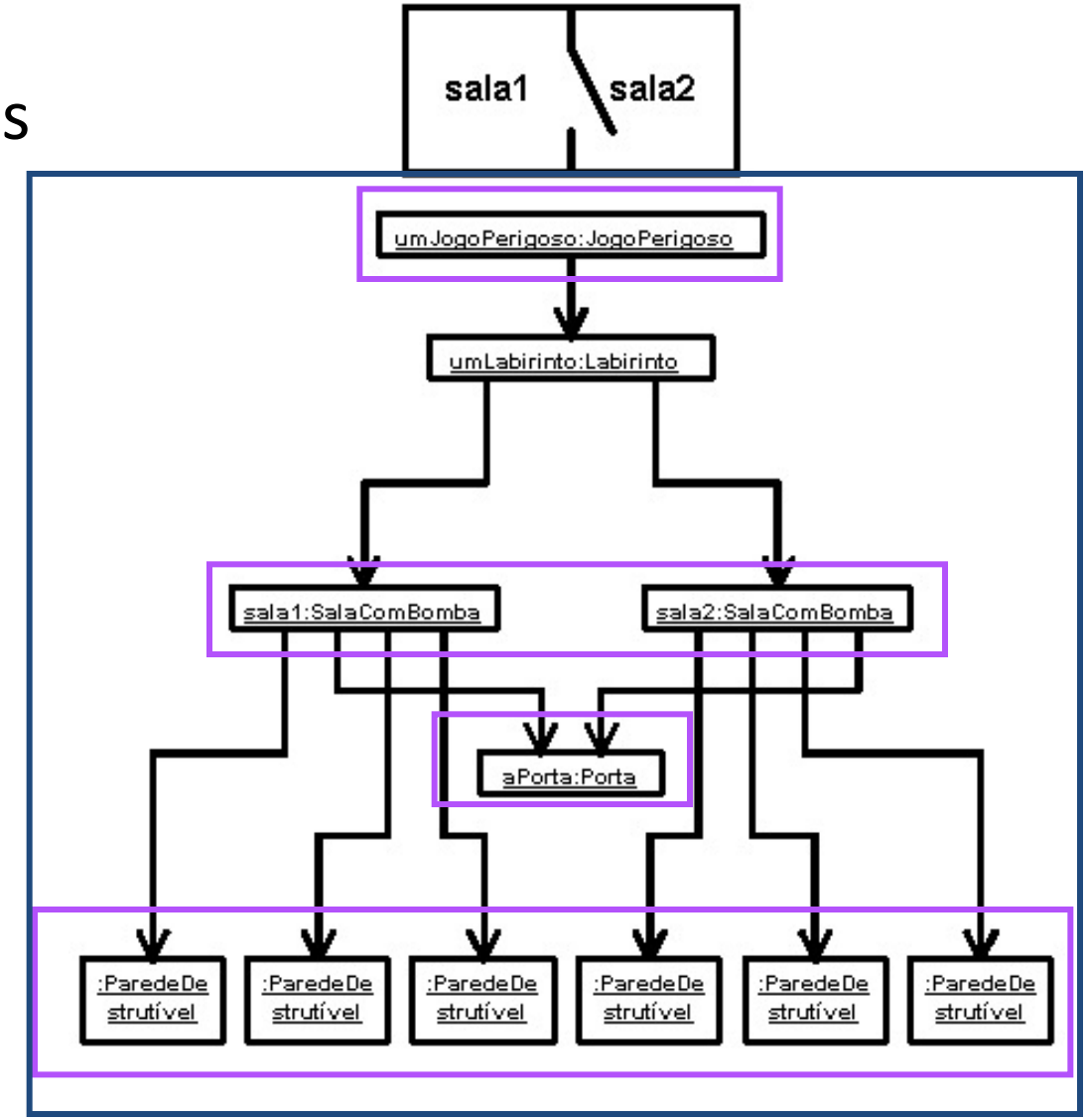
- Diagrama de Objetos

O que muda?

Os tipos

O que Não muda?

A estrutura



FACTORY METHOD

- **Colaboração**
  - Criador depende de suas subclasses para definir o Factory Method para que ele retorne uma instância do ProdutoConcreto apropriado

FACTORY METHOD

- **Conseqüências**
  - Factory Methods eliminam a necessidade de colocar classes específicas da aplicação no código
    - O código só lida com a interface Produto
    - O código pode portanto funcionar com qualquer classe ProdutoConcreto
  - Provê ganchos para subclasses
    - Criar objetos dentro de uma classe com um Factory Method é sempre mais flexível do que criar objetos diretamente
    - O Factory Method provê um gancho para que subclasses forneçam uma versão estendida de um objeto

FACTORY METHOD

- Quando usar?
  - Quando uma classe (o criador) não pode antecipar a classe dos objetos que deve criar
    - Definição dos tipos em tempo de execução
  - Quando uma classe quer que suas subclasses especifiquem os objetos criados

Dúvida?